🐳 Complete Docker Roadmap: From Basics to Job-Ready Mastery

Last Updated: April 2026
Target Audience: Absolute Beginners → DevOps Engineers & Architects
Expected Duration: 4-6 months of structured learning

📋 Table of Contents

1. Phase 1: Docker Fundamentals (Weeks 1-3)
2. Phase 2: Container Operations (Weeks 4-6)
3. Phase 3: Images & Registries (Weeks 7-9)
4. Phase 4: Docker Networking (Weeks 10-12)
5. Phase 5: Storage & Volumes (Weeks 13-15)
6. Phase 6: Docker Compose (Weeks 16-18)
7. Phase 7: Production Deployment (Weeks 19-21)
8. Phase 8: Advanced Orchestration (Weeks 22-24)
9. Phase 9: Security & Best Practices (Weeks 25-27)
10. Phase 10: Real-World Projects & Capstone (Weeks 28-30)

Phase 1: Docker Fundamentals (Weeks 1-3)

1.1 What is Docker?

- **Containerization vs Virtualization**: Understanding the difference between containers and VMs
  - VMs: Full OS per application (heavy, slow startup)
  - Containers: Lightweight, isolated processes sharing kernel
- **Why Docker?**: Speed, consistency, scalability, developer experience
- **Docker Architecture Overview**:
  - Docker Engine (Docker Daemon, REST API, CLI)
  - Docker Client (command-line interface)
  - Registries (Docker Hub, private registries)
  - Images (blueprints)
  - Containers (running instances)
- **Use Cases**: Microservices, CI/CD pipelines, testing, local development parity

1.2 Installation & Environment Setup

- **Windows Installation:**
  - Docker Desktop for Windows (WSL 2 backend recommended)
  - System requirements (virtualization enabled, 4GB+ RAM)
  - GPU support configuration
- **macOS Installation:**
  - Docker Desktop for Mac (Intel vs Apple Silicon considerations)
  - Resource allocation (CPU, RAM, disk space)
  - Virtualization framework
- **Linux Installation:**
  - Installation via package managers (apt, yum)
  - Post-installation setup (user groups, systemd service)
  - Building from source (optional)
- **Verifying Installation:** `docker --version`, `docker run hello-world`
- **Docker Configuration:** daemon.json, system resources, logging drivers

1.3 Core Concepts

- **Images:** Read-only templates containing application code, dependencies, runtime
  - Layers: How images are constructed
  - Tags and versions
  - Base images
- **Containers:** Running instances of images
  - Isolation and namespaces
  - Resource limits
  - Lifecycle: create, start, run, stop, remove
- **Registries:** Centralized repositories for images
  - Docker Hub (public)
  - Private registries
  - Registry authentication
- **Docker Hub Basics:**
  - Creating an account
  - Exploring official images
  - Understanding image naming conventions (user/repo:tag)

1.4 Your First Container

- **Running basic containers**:
  - `docker run nginx`
  - Understanding flags: `-d`, `-it`, `-p`, `-e`, `--name`
  - Interactive vs detached modes
- **Hello-world dissection**: Tracing what happens step by step
- **Running different images**:
  - Alpine Linux for minimal footprint
  - Ubuntu/Debian for full functionality
  - Official images vs community images
- **Viewing logs**: `docker logs`, `docker logs -f` (follow)
- **Inspecting containers**: `docker ps`, `docker inspect`

1.5 Common Misconceptions & Troubleshooting

- Containers are NOT lightweight VMs
- Each container runs one main process (best practice)
- Understanding `ENTRYPOINT` vs `CMD`
- Common errors and solutions:
  - "Cannot connect to Docker daemon"
  - "Port already in use"
  - "Image not found"

Phase 2: Container Operations (Weeks 4-6)

2.1 Container Lifecycle Management

- **Container States**:
  - Created, Running, Paused, Stopped, Exited, Deleted
  - State transitions and commands
- **Starting & Stopping**:
  - `docker start`: Restart a stopped container
  - `docker stop`: Graceful shutdown (SIGTERM)
  - `docker kill`: Force shutdown (SIGKILL)
  - Timeout handling

- **Pausing & Unpausing:**
  - `docker pause`: Freeze processes
  - `docker unpause`: Thaw processes
  - Use cases (resource management, debugging)
- **Removing Containers:**
  - `docker rm`: Remove stopped containers
  - `docker rm -f`: Force remove running containers
  - Cleanup with `docker container prune`
- **Restarting Policies:**
  - `--restart=no`: Don't restart
  - `--restart=always`: Always restart
  - `--restart=unless-stopped`: Restart unless explicitly stopped
  - `--restart=on-failure`: Restart on failure with retry limits

2.2 Accessing & Executing Commands

- **Interactive Shell Access:**
  - `docker exec -it container_id /bin/bash`
  - `docker run -it image /bin/bash` (create new)
  - Shell vs sh in Alpine
- **Running One-off Commands:**
  - `docker exec container_name command`
  - Use cases (debugging, database migrations, backups)
- **Port Mapping & Exposure:**
  - `-p host_port:container_port` (expose and map)
  - `-P` (map all exposed ports randomly)
  - Understanding port binding
  - `docker port` command
- **Viewing Container Processes:**
  - `docker top`: View running processes
  - `docker stats`: Real-time resource usage
  - Memory limits and CPU constraints
- **Container Inspection:**
  - `docker inspect`: Complete metadata (JSON format)
  - Filtering: `docker inspect --format='{{.State}}'`

- Network settings, mounts, environment variables

2.3 Working with Environment Variables

- **Setting Environment Variables**:
  - `-e KEY=VALUE` flag
  - `-e` vs `--env-file`
  - Multiple environment variables
- **.env Files**:
  - Creating and organizing `.env` files
  - Security considerations (don't commit secrets)
  - Using `--env-file` flag
- **Viewing Environment**:
  - `docker inspect` to view env vars
  - `docker run -e` with no value inherits from host
- **Environment Variable Expansion**:
  - How applications use environment variables
  - Common patterns in popular images (Node, Python, etc.)

2.4 Container Limits & Resource Management

- **CPU Limits**:
  - `--cpus`: Limit CPU cores (e.g., --cpus="0.5")
  - `--cpuset-cpus`: Pin to specific cores
  - Understanding CPU shares and guarantees
- **Memory Limits**:
  - `--memory`: Set memory limit (e.g., --memory=512m)
  - Out of memory (OOM) behavior
  - `--memory-swap`, `--memswap-limit`
- **Device Limits**:
  - `--blkio-weight`: Block I/O weight
  - `--device-read-bps`, `--device-write-bps`: I/O rate limiting
- **Resource Monitoring**:
  - `docker stats`: Live resource metrics
  - `docker stats --no-stream`: One-time snapshot

    - Understanding % vs absolute values
- **Resource Requests vs Limits** (Swarm/Kubernetes concepts):
    - Reservation (guaranteed minimum)
    - Limit (hard cap)

2.5 Container Naming & Identification

- **Container IDs**:
    - Long ID (sha256) vs short ID
    - Using IDs in commands
- **Container Names**:
    - `--name` flag when creating
    - Requirements: lowercase, numbers, hyphens, underscores
    - Uniqueness enforcement
- **Image Naming Convention**:
    - `[registry]/[namespace]/[repository]:[tag]`
    - Default registry (Docker Hub)
    - Tag best practices
- **Listing & Filtering**:
    - `docker ps`: Running containers
    - `docker ps -a`: All containers (including stopped)
    - Filtering by status, name, label: `docker ps --filter`
    - Formatting output: `--format`

Phase 3: Images & Registries (Weeks 7-9)

3.1 Understanding Docker Images in Depth

- **Image Layers**:
    - Filesystem layers as read-only snapshots
    - How layers are reused (efficiency)
    - Layer caching during builds
    - Using `docker history` to inspect layers
- **Image IDs & Digests**:
    - SHA256 digest (content-addressable)

    - Image ID vs digest
    - Reproducibility and immutability
- **Official Images**:
    - Where to find them (Docker Hub official library)
    - Quality and maintenance standards
    - Regular updates and security patches
- **Image Tagging Strategy**:
    - Semantic versioning (v1.2.3)
    - Latest vs specific tags
    - Multiple tags for same image
    - Tag immutability in production
- **Image Size Optimization** (covered deeper in Phase 3.3):
    - Alpine vs full images
    - Multistage builds
    - Layer caching strategies

3.2 Building Custom Images with Dockerfile

- **Dockerfile Basics**:
    - Anatomy of a Dockerfile (line by line)
    - Comment syntax (#)
    - Instruction case-insensitivity (best practice: uppercase)
- **Core Instructions**:
    - `FROM`: Base image selection (always first, except ARG)
    - `RUN`: Execute commands (create layers)
    - `COPY`: Copy files from host
    - `ADD`: Copy + extract + URL support (use COPY by default)
    - `WORKDIR`: Set working directory
    - `EXPOSE`: Document exposed ports (informational)
    - `ENV`: Set environment variables (persistent)
    - `CMD`: Default command when container starts
    - `ENTRYPOINT`: Configure container as executable
    - `USER`: Run subsequent commands as user
    - `VOLUME`: Mark directories for persistence
    - `LABEL`: Add metadata (key=value pairs)

    - `SHELL`: Change shell (especially for Windows)
- **CMD vs ENTRYPOINT**:
    - CMD as default arguments
    - ENTRYPOINT as entry point
    - Combined usage: ENTRYPOINT ["python"] + CMD ["app.py"]
    - Override behavior with docker run arguments
- **Build Context**:
    - What gets sent to Docker daemon
    - `.dockerignore` file (like .gitignore)
    - Optimizing context size
- **Building Images**:
    - `docker build -t image_name:tag .`
    - `--build-arg`: Pass build-time arguments
    - `--target`: Multistage build target
    - Build process output and layer caching
- **Best Practices for Dockerfiles**:
    - Minimal layers (combine RUN commands)
    - Specific base image tags (avoid `latest`)
    - Layer ordering (frequently changing last)
    - Using .dockerignore for context optimization
    - Principle of least privilege (non-root user)

3.3 Advanced Image Optimization

- **Multistage Builds**:
    - Building in one stage, copying artifacts to final
    - Example: Build Go binary, copy to Alpine
    - Reducing final image size dramatically
    - `FROM ... AS stage_name`
    - `COPY --from=stage_name`
- **Base Image Selection**:
    - `scratch`: Empty image (for static binaries)
    - `alpine`: Minimal (5-15MB)
    - `debian:slim`, `ubuntu:focal`: Medium (100-200MB)
    - `debian`, `ubuntu`: Full (200-500MB)

    - Trade-offs: Size vs compatibility vs tools
- **Caching & Build Performance**:
    - How Docker caches layers
    - When cache is invalidated
    - Ordering instructions for optimal cache hit
    - `--no-cache` flag
    - BuildKit for advanced caching
- **BuildKit Features** (modern Docker build):
    - `DOCKER_BUILDKIT=1 docker build`
    - Parallel builds
    - Enhanced caching
    - Progress output
- **Security in Images**:
    - Scanning for vulnerabilities
    - Using specific image digests (not tags)
    - Minimal images reduce attack surface
    - Non-root user principle

3.4 Docker Registries

- **Docker Hub**:
    - Public and private repositories
    - Official images
    - Community images
    - Stars, pulls, documentation
- **Authentication & Login**:
    - `docker login` (Docker Hub)
    - `docker login registry_url` (private registry)
    - Credentials storage (secure token, config.json)
- **Pushing Images**:
    - `docker tag image_id registry/repo:tag`
    - `docker push registry/repo:tag`
    - Push workflow (layer upload)
- **Pulling Images**:
    - `docker pull registry/repo:tag`

    - Default registry behavior
    - Specifying custom registries
- **Private Registries**:
    - Self-hosted options: Docker Registry, Nexus, Harbor
    - Container Registry as a Service (AWS ECR, GCP GCR, Azure ACR)
    - Authentication and authorization
- **Image Naming Conventions**:
    - Fully qualified name: `registry.example.com/team/app:v1.0`
    - Repository naming best practices
    - Tag immutability and versioning strategy

3.5 Image Management

- **Listing Images**: `docker images`, `docker image ls`
- **Searching Images**: `docker search` (limited, better use web interface)
- **Inspecting Images**:
    - `docker inspect image_id`: Metadata (JSON)
    - `docker history image_id`: Layer history
    - Filtering and formatting output
- **Removing Images**:
    - `docker rmi image_id`: Remove image
    - `docker image prune`: Clean up unused images
    - Dangling images and orphans
- **Image Tagging**:
    - `docker tag source_image:tag target_image:tag`
    - Creating aliases
    - Retagging and versioning

Phase 4: Docker Networking (Weeks 10-12)

4.1 Docker Networking Fundamentals

- **Network Types**:
    - Bridge (default, isolated network per host)
    - Host (use host's network stack)
    - None (no networking)

    - Overlay (Swarm mode, multi-host)
    - Macvlan (assign MAC addresses)
- **Default Bridge Network:**
    - Containers on default bridge
    - Automatic DNS resolution NOT available
    - Not recommended for production multi-container
- **User-Defined Bridge Networks:**
    - Creating custom networks
    - Containers on same network can resolve by name
    - Better isolation and control
    - Network driver options
- **Network Isolation:**
    - Containers on different networks cannot communicate by default
    - Containers on same network can communicate
    - Published ports for external access

4.2 Container Communication

- **Port Mapping (Publishing):**
    - `-p host_port:container_port`
    - `0.0.0.0:host_port:container_port` (all interfaces)
    - `127.0.0.1:host_port:container_port` (localhost only)
    - UDP support: `-p 53:53/udp`
    - `docker port` command to view mappings
- **Service Discovery:**
    - DNS by container name (user-defined bridge)
    - Embedded DNS server (127.0.0.11:53)
    - Hostname and domain options
    - Links (legacy, deprecated approach)
- **Container-to-Container Communication:**
    - Direct IP addressing
    - DNS name resolution
    - Exposing ports (--expose flag)
- **Network Aliases:**
    - Multiple names for same container

- `--network-alias` flag
- Round-robin with multiple containers

4.3 Advanced Networking

- **Custom Network Creation:**
  - `docker network create my_network`
  - `--driver bridge` (default)
  - `--subnet`, `--gateway` (IP configuration)
  - Custom IPAM (IP Address Management)
- **Connecting Containers to Networks:**
  - `docker network connect network_name container_name`
  - `docker network disconnect`
  - Container on multiple networks
  - Changing networks (advanced patterns)
- **Host Network Mode:**
  - `--network host`
  - Container shares host's network stack
  - Performance benefit, no isolation
  - Linux-specific (limited on Windows/Mac)
  - When to use (high-performance scenarios)
- **Overlay Networks** (Swarm mode):
  - Multi-host networking
  - Encryption options
  - Service load balancing
- **Network Troubleshooting:**
  - `docker network ls`: List networks
  - `docker network inspect network_name`: Network details
  - `docker network connect/disconnect`: Manage connections
  - Debugging tools: ping, curl, netstat within containers

4.4 Exposing Services

- **EXPOSE Instruction:**
  - Metadata for documentation
  - Does NOT publish ports

    - Multiple ports in Dockerfile
- **Port Publishing**:
    - `-p` when running container
    - Publishing multiple ports
    - Random port assignment with `-P`
    - Port ranges
- **Load Balancing**:
    - Swarm mode load balancing (ingress mode)
    - External load balancers
    - Sticky sessions and persistence

4.5 DNS & Service Discovery

- **DNS Resolution**:
    - Container's `/etc/resolv.conf`
    - nameserver pointing to embedded DNS
    - Search domains
- **Service Names**:
    - Hostname == container name (user-defined bridge)
    - Full domain: `container_name.network_name`
    - Aliases for service discovery
- **Round-robin DNS** (Swarm):
    - Multiple containers with same name
    - Load distribution

Phase 5: Storage & Volumes (Weeks 13-15)

5.1 Data Persistence Basics

- **The Problem**:
    - Container filesystems are ephemeral
    - Data lost when container removed
    - Need for persistent data
- **Types of Data**:
    - Stateless (app code) - no persistence needed
    - Stateful (databases, user uploads) - persistence critical

    - Logs and metrics (semi-persistent)
- **Storage Options**:
    - Volumes (managed by Docker)
    - Bind mounts (host directory mount)
    - tmpfs mounts (in-memory)
- **Use Case Selection Guide**:
    - Volumes: Databases, important application data
    - Bind mounts: Development, debugging, sharing code
    - tmpfs: Temporary cache, secrets (with caution)

5.2 Volumes in Depth

- **Volume Creation**:
    - `docker volume create my_volume`
    - Named volumes (referenced by name)
    - Anonymous volumes (created with container, auto-cleanup)
    - `--rm` flag with anonymous volumes
- **Mounting Volumes**:
    - `-v volume_name:/path/in/container`
    - Read-only volumes: `-v volume_name:/path:ro`
    - Read-write (default): `-v volume_name:/path:rw`
    - Multiple volumes per container
- **Volume Drivers**:
    - Local driver (default, host filesystem)
    - Third-party drivers (NFS, cloud providers)
    - `--driver` flag during creation
    - Driver options: `--opt`
- **Volume Management**:
    - `docker volume ls`: List volumes
    - `docker volume inspect`: Volume details and mountpoint
    - `docker volume rm`: Remove volumes
    - `docker volume prune`: Clean up unused volumes
- **Data Backup & Migration**:
    - Backup: `docker run -v volume:/backup -v /backups:/backup ubuntu tar czf /backup/volume.tar.gz -C / backup`

- Restore: Extract from backup
- Cross-host migration strategies

5.3 Bind Mounts

- **Bind Mount Mechanics:**
  - `-v /host/path:/container/path`
  - Full paths required (no relative paths)
  - Host path must exist
- **Bind Mount Use Cases:**
  - Development (live code editing)
  - Configuration files
  - Sharing files between containers
  - Access to host resources
- **Bind Mount in Docker Compose:**
  - Relative paths supported
  - Expanding to absolute paths
  - SELinux and AppArmor labels
- **Performance Considerations:**
  - Bind mounts slower than volumes (especially Mac/Windows)
  - File sync issues on Windows
  - Cached, delegated options
- **Permissions & Ownership:**
  - File permissions (uid/gid)
  - Container user must have access
  - Ownership issues and solutions
  - `--user` flag interaction

5.4 Temporary Storage & tmpfs

- **tmpfs Mounts:**
  - `--tmpfs /path` flag
  - In-memory filesystem
  - Not persisted to disk
  - Fast, volatile storage
- **Use Cases:**

    - Temporary caches
    - Session storage
    - Secrets (careful with security)
    - PID and IPC namespaces
- **Size Limits:** `--tmpfs /path:size=500m`
- **Comparison with volumes:**
    - Speed advantage
    - Volatility (lost on container stop)
    - Security implications

5.5 Advanced Storage Patterns

- **Multi-container Volumes:**
    - Data containers pattern (legacy)
    - Volume sharing between containers
    - Data-only containers
- **Database Considerations:**
    - Persistent volume for database data
    - Mount point placement
    - Performance tuning
- **Log Management:**
    - Docker's logging drivers
    - Volume-based log collection
    - Log rotation and retention
- **Health Checks with Persistence:**
    - Combining volumes with health checks
    - Recovery from failures
    - Data consistency

Phase 6: Docker Compose (Weeks 16-18)

6.1 Docker Compose Fundamentals

- **What is Docker Compose?**
    - Tool for defining multi-container applications
    - Single YAML file definition

    - "Docker on steroids" for local development
    - Not for production orchestration (use Swarm/Kubernetes)
- **Compose File Versions**:
    - v1 (deprecated)
    - v2.x (legacy)
    - v3.x (current, Swarm-compatible)
    - v4.x (advanced features)
    - Compatibility with Docker versions
- **Installation**:
    - Bundled with Docker Desktop
    - Standalone installation (Linux)
    - Version verification: `docker-compose --version`
- **Basic Concepts**:
    - Services (containers to be created)
    - Networks (communication between services)
    - Volumes (persistent data)
    - Environment variables and configurations

6.2 Compose File Structure

- **Root-level Properties**:
    - `version`: Compose file format version
    - `services`: Define containers
    - `networks`: Define networks
    - `volumes`: Define volumes
- **Service Definition**:
    - `image`: Image to run
    - `container_name`: Name of container
    - `build`: Build image from Dockerfile
    - `ports`: Port mappings
    - `environment`: Environment variables
    - `env_file`: Load from .env file
    - `volumes`: Mount volumes/bind mounts
    - `networks`: Attach to networks
    - `command`: Override default command

    - `entrypoint`: Override entrypoint
    - `depends_on`: Service dependencies
    - `restart_policy`: Restart behavior
    - `healthcheck`: Health check definition
    - `labels`: Service metadata
    - `logging`: Logging configuration
- **Network Definition**:
    - `driver`: Network driver (bridge, overlay, etc.)
    - `ipam`: IP address management
    - Service automatic network attachment
- **Volume Definition**:
    - `driver`: Volume driver
    - `driver_opts`: Driver options
    - External volumes (pre-created)

6.3 Building with Compose

- **Using `build` Context**:
    - `build: ./myapp` (Dockerfile in directory)
    - `build: { context: ..., dockerfile: Dockerfile.prod }`
    - Build arguments: `args: { BUILD_VERSION: 1.0 }`
    - Target in multistage: `target: production`
- **Image Naming During Build**:
    - Automatic naming based on directory and service
    - Custom image name: `image: myrepo/myimage:tag` with build
    - Push after build
- **Build Caching**:
    - `docker-compose build --no-cache`
    - Cache invalidation strategies
- **Build Arguments vs Environment**:
    - ARG at build-time
    - ENV at runtime
    - Passing values to Dockerfile

6.4 Running Applications with Compose

- **Starting Services:**
  - `docker-compose up`: Start and display logs
  - `docker-compose up -d`: Start in background
  - `docker-compose start`: Start stopped services
  - Building images automatically (`--build`)
  - Pull images: `--pull always`
- **Stopping Services:**
  - `docker-compose down`: Stop and remove containers
  - `docker-compose stop`: Stop without removing
  - `docker-compose pause`: Pause services
- **Service Dependencies:**
  - `depends_on`: Define dependencies
  - Service startup order (not readiness)
  - Waiting for service readiness (custom solutions)
- **Scaling Services:**
  - `docker-compose up --scale web=3`
  - Port assignment when scaling
  - Limitations and considerations
- **Logs:**
  - `docker-compose logs`: View logs
  - `docker-compose logs -f`: Follow logs
  - `docker-compose logs service_name`: Single service
- **Executing Commands:**
  - `docker-compose exec service_name command`
  - Interactive shell: `docker-compose exec -it web bash`

6.5 Advanced Compose Features

- **Overrides and Extends:**
  - `extends`: Inherit service definition
  - Override files: `docker-compose -f base.yml -f override.yml up`
  - Multi-environment configurations (dev, test, prod)
- **Health Checks:**

    - `healthcheck: test:` Define health check
    - `interval`, `timeout`, `retries`, `start_period`
    - Dependency based on health (custom scripts)
- **Resource Limits**:
    - `resources.limits.cpus`: CPU limit
    - `resources.limits.memory`: Memory limit
    - `resources.reservations`: Requested resources
- **Restart Policies**:
    - `restart_policy: always`, `unless-stopped`, etc.
    - Max retries with exponential backoff
- **Logging Drivers**:
    - `logging: driver: json-file`
    - Limiting log size: `options: max-size`
    - Different logging drivers (splunk, awslogs, etc.)
- **Named Services & Profiles**:
    - `profiles`: Conditional service inclusion
    - `docker-compose --profile dev up`
- **Secrets & Configs** (Swarm mode):
    - Using docker secrets
    - Config management
    - Runtime injection

6.6 Development Workflows with Compose

- **Hot Reloading**:
    - Volumes for code changes
    - Application-level hot reload (Flask, Node with nodemon)
    - Database persistence during development
- **Database Management**:
    - PostgreSQL/MySQL service definition
    - Volume for data persistence
    - Initialization scripts
    - Network communication between services
- **Debugging**:
    - Exposing debug ports

    - Using debuggers with containers
    - `docker-compose exec` for inspecting services
- **Environment Management**:
    - `.env` files for local development
    - Different `.env.production` files
    - Secret management (use docker secrets in production)
- **CI/CD Integration**:
    - `docker-compose` in CI pipelines
    - Running tests with compose
    - Artifact collection

Phase 7: Production Deployment (Weeks 19-21)

7.1 Moving from Development to Production

- **Development vs Production Differences**:
    - Scale and performance requirements
    - Security hardening
    - Logging, monitoring, alerting
    - High availability and reliability
    - Resource constraints
    - Cost optimization
- **Configuration Management**:
    - Environment-specific configurations
    - Secrets management (credentials, API keys, certificates)
    - Configuration as code approach
- **Image Optimization for Production**:
    - Minimal images (Alpine base)
    - Multistage builds
    - No development dependencies
    - Security scanning and vulnerability checks
- **Documentation & Handoff**:
    - Architecture documentation
    - Deployment runbooks
    - Monitoring and alerting setup

- Disaster recovery procedures

7.2 Container Registry Strategies

- **Image Naming & Versioning**:
  - Semantic versioning (v1.0.0)
  - Git commit hash tagging
  - Build number tagging
  - Latest tag considerations (avoid in production)
  - Immutable tags for production releases
- **Private Registry Setup**:
  - Docker Registry (self-hosted)
  - Container registries as a service:
    - AWS ECR (Elastic Container Registry)
    - Google Container Registry (GCR) / Artifact Registry
    - Azure Container Registry (ACR)
    - Harbor (open-source)
  - Access control and authentication
  - Image scanning for vulnerabilities
- **Registry Mirroring**:
  - Pulling from multiple registries
  - Caching and performance
  - High-availability registry setup
- **Image Promotion Pipeline**:
  - Dev → Staging → Production flow
  - Tagging strategy
  - Image promotion automation

7.3 Logging, Monitoring & Observability

- **Container Logging**:
  - Docker logging drivers (json-file, splunk, awslogs, etc.)
  - Centralized logging (ELK, Splunk, Loki)
  - Log levels and formatting
  - Structured logging (JSON format)
  - Log aggregation from multiple containers

- **Metrics & Monitoring:**
  - Prometheus for metrics collection
  - Grafana for visualization
  - cAdvisor for container metrics
  - Custom application metrics
  - Alerting rules and thresholds
- **Distributed Tracing:**
  - Jaeger or Zipkin setup
  - Correlation IDs
  - Request tracing across services
- **Health Checks:**
  - Container health endpoints
  - Liveness vs Readiness probes
  - Graceful shutdown and SIGTERM handling
  - Health check metrics

7.4 Security Hardening

- **Image Security:**
  - Use specific base image tags (not latest)
  - Vulnerability scanning (Trivy, Anchore, Snyk)
  - Minimal images reduce attack surface
  - Signing images with content trust
- **Container Runtime Security:**
  - Running as non-root user
  - Read-only root filesystem
  - Capabilities drop (CAP_DROP)
  - Seccomp profiles
  - AppArmor/SELinux policies
  - Resource limits prevent DoS
- **Image Content:**
  - No credentials in images
  - Secrets passed at runtime (environment, mounted files)
  - Use multi-stage builds to exclude build tools
  - Scan for hardcoded secrets
- **Network Security:**

    - Network policies (Kubernetes)
    - Firewall rules
    - TLS/SSL for service communication
    - RBAC for registry access
- **Supply Chain Security**:
    - Trusted base images
    - Image signing and verification
    - Provenance tracking
    - Dependency scanning

7.5 Deployment Patterns

- **Blue-Green Deployments**:
    - Two identical production environments
    - Switch traffic between blue and green
    - Zero-downtime deployments
    - Easy rollback
- **Canary Deployments**:
    - Gradual rollout to subset of users
    - Monitor metrics during rollout
    - Automatic rollback on errors
    - Service mesh support (Istio, Linkerd)
- **Rolling Deployments**:
    - Gradually replace old containers with new
    - Maintains service availability
    - Handles graceful shutdown
- **Recreate Deployment**:
    - Stop old, start new
    - Simplest but causes downtime
    - For non-critical services
- **Feature Flags**:
    - Decouple deployment from feature enablement
    - A/B testing
    - Gradual feature rollout

Phase 8: Advanced Orchestration (Weeks 22-24)

8.1 Docker Swarm Mode

- **Swarm Basics**:
  - Single manager-worker model
  - Built into Docker Engine (no extra installation)
  - Declarative service definition
  - Automatic load balancing
  - Built-in service discovery
- **Swarm Initialization**:
  - `docker swarm init`: Initialize on first node
  - `docker swarm join-token manager/worker`: Get join tokens
  - `docker node ls`: View cluster nodes
  - Node promotion and demotion
- **Services in Swarm**:
  - `docker service create`: Create service
  - `docker service ps`: View service tasks
  - `docker service inspect`: Service details
  - Service mode: Replicated vs Global
- **Scaling Services**:
  - `docker service scale service_name=5`
  - Rolling updates
  - Update configuration: `--update-parallelism`, `--update-delay`
- **Service Discovery & Load Balancing**:
  - Ingress mode (external load balancing)
  - Host mode (direct port binding)
  - Internal DNS
  - VIP (Virtual IP) for service
- **Secrets Management** (Swarm):
  - `docker secret create`: Create secret
  - Mount secrets in services
  - Encrypted in transit and at rest
  - Non-accessible to other services

- **Configs** (Swarm):
  - `docker config create`: Create config
  - Configuration files for services
  - Update configurations without rebuilding images

8.2 Introduction to Kubernetes

- **Why Kubernetes?**
  - Industry standard for container orchestration
  - Large-scale deployments
  - Complex networking and storage
  - Self-healing and auto-scaling
  - Rich ecosystem
- **Kubernetes vs Docker Swarm**:
  - Complexity vs simplicity trade-off
  - Kubernetes for production at scale
  - Swarm for smaller deployments
- **Kubernetes Architecture**:
  - Control plane (API server, scheduler, controller-manager)
  - Nodes (worker machines)
  - Pods (smallest deployable unit, not containers)
  - Services (expose pods)
  - Namespaces (virtual clusters)
- **Local Kubernetes Setup**:
  - Minikube (single-node cluster)
  - kind (Kubernetes in Docker)
  - Docker Desktop Kubernetes
- **Basic Kubernetes Resources**:
  - Pod: Wrapper around container
  - Deployment: Manage pods
  - Service: Expose pods to network
  - ConfigMap: Configuration data
  - Secret: Sensitive data
  - PersistentVolume: Storage
  - Ingress: HTTP routing

8.3 Container Runtimes Beyond Docker

- **Alternative Runtimes:**
  - containerd (CNCF-graduated, used by Kubernetes)
  - runc (OCI runtime spec implementation)
  - CRI-O (Kubernetes-focused)
  - Podman (Docker-compatible, no daemon)
- **OCI Standards:**
  - Open Container Initiative
  - Image spec and runtime spec
  - Standard-compliant image format
  - Compatibility across tools
- **Docker vs Podman:**
  - Daemon vs daemonless architecture
  - API compatibility
  - Pod support in Podman
  - Use case differences

8.4 Container Networking in Production

- **Service Mesh:**
  - Istio for advanced networking
  - Linkerd for simplicity
  - Observability and traffic management
  - Circuit breaking and retries
  - mTLS for secure communication
- **Ingress Controllers:**
  - Expose services to external traffic
  - SSL/TLS termination
  - Path-based routing
  - Host-based routing
- **Network Policies:**
  - Restrict traffic between pods/containers
  - Whitelist-based approach
  - Implement zero-trust networking
- **Load Balancing Strategies:**

- Round-robin
- Session affinity
- Geographic routing
- Health-based routing

8.5 High Availability & Disaster Recovery

- **Redundancy**:
  - Multiple replicas of services
  - Multi-region deployments
  - Backup and restore procedures
- **Failover & Recovery**:
  - Automatic failover
  - Health-based replacement
  - Graceful shutdown handling
  - Leader election (for stateful services)
- **Backup Strategies**:
  - Regular database backups
  - Volume snapshots
  - Infrastructure as Code (backup and restore)
  - Backup testing and validation
- **Disaster Recovery Plan**:
  - RTO (Recovery Time Objective)
  - RPO (Recovery Point Objective)
  - Regular DR drills
  - Documentation and runbooks

Phase 9: Security & Best Practices (Weeks 25-27)

9.1 Container Image Security

- **Image Scanning**:
  - Vulnerability scanning tools:
    - Trivy (lightweight, open-source)
    - Anchore (comprehensive, policy-based)
    - Snyk (developer-focused, integrates with registries)

        - Clair (CNCF, for registries)
    - Scanning in CI/CD pipelines
    - Baseline images for scanning
    - Fixing vulnerabilities (patching vs updating)
- **Base Image Selection**:
    - Official images maintenance and support
    - Distroless images (Google)
    - Alpine vs other minimal options
    - Signing and verifying base images
- **Software Bill of Materials (SBOM)**:
    - Generating SBOM
    - syft tool for SBOM generation
    - Tracking dependencies
    - License compliance
- **Image Signing & Verification**:
    - Docker Content Trust
    - Notary for image signing
    - Cosign (CNCF, modern approach)
    - Signature verification in pull process
- **Supply Chain Security**:
    - Provenance tracking
    - Signed commits
    - Artifact provenance (SLSA framework)
    - Secure build pipelines

9.2 Runtime Security

- **Principle of Least Privilege**:
    - Running as non-root user
    - Minimal capabilities (CAP_DROP)
    - Read-only root filesystem where possible
    - Minimal image contents
- **Seccomp Profiles**:
    - System call filtering
    - Default Docker seccomp profile

    - Custom seccomp profiles
    - RuntimeDefault vs Unconfined
- **AppArmor & SELinux**:
    - Mandatory access control
    - Container-specific policies
    - Labeling systems
    - Enforcement modes
- **Resource Limits & Quotas**:
    - CPU and memory limits prevent DoS
    - Disk space quotas
    - File descriptor limits
    - Network bandwidth limits
- **Container Capabilities**:
    - Linux capabilities
    - Dropping unnecessary capabilities
    - CAP_NET_BIND_SERVICE, CAP_SYS_ADMIN, etc.
    - Capability-based security model

9.3 Secret Management

- **Secrets in Containers**:
    - Never commit secrets to images
    - Environment variables (flexible, not ideal for large secrets)
    - Mounted files (cleaner, file-based)
    - Secret management tools
- **Container-Level Secrets**:
    - Docker Swarm secrets
    - Kubernetes secrets
    - Rotating secrets
    - Secret access logging
- **External Secret Management**:
    - HashiCorp Vault
    - AWS Secrets Manager
    - Azure Key Vault
    - Google Cloud Secret Manager

    - Sealed Secrets (Kubernetes)
- **Secret Rotation**:
    - Regular rotation schedules
    - Rolling updates to apply new secrets
    - Graceful handling of secret changes
- **Secret Scanning**:
    - Scanning code repositories for leaked secrets
    - Tools: git-secrets, TruffleHog, detect-secrets
    - Prevention in CI/CD pipelines

9.4 Network Security

- **Container Isolation**:
    - Network namespaces
    - User namespaces for UID mapping
    - IPC namespaces
    - PID namespaces
- **Network Policies**:
    - Default-deny approach
    - Whitelist-based rules
    - Ingress and egress policies
    - Labels and selectors
- **TLS/SSL Communication**:
    - Encrypting inter-container communication
    - Certificate management and rotation
    - mTLS for mutual authentication
- **Firewall & WAF**:
    - Host-level firewall rules
    - Container network firewall
    - Web Application Firewall for applications
    - DDoS protection

9.5 Compliance & Auditing

- **Container Compliance Scanning**:
    - CIS Docker Benchmark

    - Tools: Aqua trivy, anchore
    - Automatic remediation
- **Audit Logging**:
    - Docker daemon audit logs
    - Container activity logging
    - Image build audit trails
    - Kubernetes audit logs
- **Compliance Standards**:
    - PCI-DSS for payment processing
    - HIPAA for healthcare
    - SOC 2 for service providers
    - GDPR for data privacy
- **Immutable Containers**:
    - Prevent runtime modifications
    - Enforce through admission controllers
    - Security scanning + read-only filesystems
- **Regular Security Updates**:
    - Patch management strategy
    - Zero-day handling
    - Testing updates in staging
    - Automated updates with proper testing

9.6 Logging & Monitoring Security

- **Security Log Aggregation**:
    - Centralized logging for all containers
    - Tamper-proof log storage
    - Long-term log retention
- **Anomaly Detection**:
    - Behavioral analysis of containers
    - Unusual process execution
    - Unexpected network connections
    - Resource consumption anomalies
- **Intrusion Detection**:
    - Falco for runtime security

    - Detecting privilege escalation
    - File integrity monitoring
    - Kernel module loading detection
- **Alerting**:
    - Real-time security alerts
    - Escalation procedures
    - Incident response automation
    - Integration with SIEM systems

Phase 10: Real-World Projects & Capstone (Weeks 28-30)

10.1 Project 1: Multi-tier Web Application

Objective: Build a complete application with frontend, backend, database

Components:

- Frontend: React/Vue container
- Backend API: Node.js/Python/Go container
- Database: PostgreSQL container
- Redis cache container
- Nginx reverse proxy

Requirements:

- Docker Compose configuration for entire stack
- Environment variable management
- Volume persistence for database
- Network communication between services
- Health checks for all services
- Resource limits

Learning Outcomes:

- Service orchestration with Compose
- Multi-container debugging
- Volume and networking concepts
- Configuration management
- Proper logging setup

10.2 Project 2: CI/CD Pipeline

Objective: Automate building, testing, and deploying containers

Pipeline Stages:

- Trigger on Git push
- Build Docker image
- Run tests in container
- Scan image for vulnerabilities
- Push to private registry
- Deploy to staging
- Run integration tests
- Deploy to production (manual approval)
- Rollback capability

Technologies:

- Git (GitHub/GitLab/Gitea)
- GitHub Actions / GitLab CI / Jenkins
- Private Docker Registry (Harbor/ECR)
- Docker Swarm or Kubernetes for deployment
- Test framework (Jest, pytest, etc.)
- Image scanning (Trivy)

Learning Outcomes:

- Containerized CI/CD workflow
- Registry and image management
- Deployment automation
- Quality gates and testing
- Version control integration

10.3 Project 3: Microservices Architecture

Objective: Build a distributed application with multiple interdependent services

Services (Minimal):

- User Service (authentication, user data)
- Product Service (product catalog)
- Order Service (order management)

- Payment Service (payment processing)
- Notification Service (email/SMS)
- API Gateway (routes requests)

Architecture:

- Each service in separate container
- Databases per service (polyglot persistence)
- Message queue (RabbitMQ/Redis) for async communication
- Service mesh or simplified networking
- Centralized logging
- Distributed tracing

Requirements:

- Service-to-service communication
- Database per service pattern
- Transaction handling (sagas)
- Failure handling and retries
- Service discovery
- Circuit breaker pattern
- API documentation

Learning Outcomes:

- Microservices design patterns
- Distributed system challenges
- Service communication
- Data consistency
- Operational complexity

10.4 Project 4: Stateful Applications

Objective: Deploy applications with persistent state

Components:

- Database (PostgreSQL or MongoDB)
- Cache (Redis)
- Message queue (RabbitMQ)
- File storage (volume or S3)

- Session store

Challenges:

- Data persistence across updates
- Backup and recovery
- Database migrations in containers
- Cluster coordination
- High availability setup

Requirements:

- Automated backups
- Point-in-time recovery
- Data validation
- Replication setup (if applicable)
- Monitoring and alerts

Learning Outcomes:

- Stateful container management
- Data persistence strategies
- Backup and disaster recovery
- High availability patterns
- Storage optimization

10.5 Project 5: Container Security & Hardening

Objective: Build a secure, production-ready containerized application

Security Measures:

- Minimal base image (distroless or Alpine)
- Multi-stage build to exclude build tools
- Non-root user
- Read-only root filesystem
- Dropped capabilities
- Seccomp profile
- Resource limits
- No hardcoded secrets
- Image scanning and signing

- Network policies

Deployment:

- Private registry with authentication
- Secure secret management
- TLS/SSL for communication
- Audit logging
- Access control and RBAC
- Regular security updates

Requirements:

- Security scanning in CI/CD
- Vulnerability remediation process
- Compliance checklist
- Security documentation
- Incident response plan

Learning Outcomes:

- Security best practices
- Threat modeling for containers
- Compliance and auditing
- Secure secrets management
- Security operations

10.6 Capstone Project: Complete Production System

Objective: Design and deploy a complete, production-ready system

Requirements:

- Realistic application (multi-tier, microservices)
- Full CI/CD pipeline
- Infrastructure automation (Docker Compose, Swarm, or Kubernetes manifests)
- Comprehensive monitoring and logging
- Security hardening
- Disaster recovery plan
- Documentation and runbooks
- Cost optimization

- Performance optimization

Deliverables:

1. **Architecture Documentation**
   - System design diagram
   - Service descriptions
   - Data flow
   - Security boundaries
   - Deployment architecture
2. **Implementation**
   - Dockerfiles (optimized, multi-stage)
   - Compose or orchestration files
   - CI/CD configuration
   - Infrastructure as Code
3. **Operations**
   - Deployment runbook
   - Monitoring and alerting setup
   - Backup and recovery procedures
   - Scaling procedures
   - Troubleshooting guide
4. **Security**
   - Security audit checklist
   - Secret management strategy
   - Network policies
   - Compliance verification
5. **Performance**
   - Load testing results
   - Resource optimization metrics
   - Scaling behavior
   - Optimization recommendations

Success Criteria:

- Application runs reliably
- Automated deployment pipeline
- Logs and metrics collected

- Security standards met
- Complete documentation
- Team can operate without author guidance
- Handles failures gracefully
- Cost-effective operation

Learning Path Summary

Phase	Duration	Focus	Key Skills
1	3 weeks	Fundamentals	Containers, images, basic commands
2	3 weeks	Operations	Lifecycle, exec, resource management
3	3 weeks	Images & Registries	Dockerfiles, building, registries
4	3 weeks	Networking	Port mapping, DNS, service discovery
5	3 weeks	Storage	Volumes, bind mounts, persistence
6	3 weeks	Compose	Multi-container applications
7	3 weeks	Production	Deployment, logging, monitoring
8	3 weeks	Orchestration	Swarm, K8s intro, container runtimes
9	3 weeks	Security	Image scanning, runtime security, secrets
10	3 weeks	Capstone	Integrated projects and production system

Total Duration: 30 weeks (7.5 months) of focused learning

Resources & Tools

Official Documentation

- Docker Documentation: https://docs.docker.com
- Docker Hub: https://hub.docker.com
- Play with Docker: https://labs.play-with-docker.com (free online labs)

Learning Platforms

- Docker's Official Training: https://www.docker.com/

- Linux Academy / A Cloud Guru Docker courses
- Udemy courses on Docker
- YouTube channels (TechWorld with Nana, Fireship, NetworkChuck)

Tools & Software

- Docker Desktop (Windows, macOS)
- Docker Engine (Linux)
- Docker Compose (included in Desktop)
- Minikube or kind (local Kubernetes)
- VS Code with Docker extension
- Trivy (vulnerability scanning)
- Docker Scout (image analysis)

Community & Support

- Docker Community Slack
- Stack Overflow (docker tag)
- Docker Forums
- GitHub issues and discussions
- Local Docker Meetups

Certification Paths

Official Docker Certifications

1. **Docker Associate (DCA)**
    - Prerequisite: Understanding of Docker fundamentals
    - Exam: 120 questions in 90 minutes
    - Covers: Images, containers, networking, storage, orchestration
    - Preparation: 6-12 months of hands-on experience
2. **Docker Certified Associate (Advanced)**
    - Higher-level certification
    - Focus on production deployments
    - Advanced orchestration concepts

Kubernetes Certifications (Next Steps)

- CKA (Certified Kubernetes Administrator)
- CKAD (Certified Kubernetes Application Developer)
- CKS (Certified Kubernetes Security Specialist)

Job Readiness Checklist

Before claiming "job-ready Docker expertise," ensure you can:

Fundamentals

- [ ] Explain containerization vs virtualization
- [ ] Write production-grade Dockerfiles
- [ ] Understand image layers and caching
- [ ] Manage container lifecycle
- [ ] Configure networking between containers

Operations

- [ ] Orchestrate multi-container apps (Compose)
- [ ] Implement persistent storage patterns
- [ ] Set up health checks and monitoring
- [ ] Implement logging aggregation
- [ ] Troubleshoot container issues

Production

- [ ] Optimize images for production
- [ ] Implement CI/CD with containers
- [ ] Secure containers and images
- [ ] Manage secrets securely
- [ ] Design high-availability systems

Advanced

- [ ] Deploy to container orchestration (Swarm/K8s)
- [ ] Implement security hardening
- [ ] Design microservices architectures
- [ ] Performance optimize containerized apps
- [ ] Implement disaster recovery

Soft Skills

- [ ] Document architecture decisions

- [ ] Troubleshoot under pressure
- [ ] Collaborate with team members
- [ ] Keep up with container ecosystem changes

Avoiding Common Pitfalls

1. **Running as Root**: Always use non-root users
2. **Large Images**: Use multistage builds and Alpine base images
3. **Ignoring Security**: Scan images, update base images regularly
4. **Premature Optimization**: Focus on correctness first
5. **Over-Composing**: Know when to move to K8s
6. **Ignoring Logs**: Implement proper logging from day one
7. **Hardcoded Configuration**: Use environment variables and secrets management
8. **No Health Checks**: Always implement health checks
9. **Unversioned Images**: Use specific, immutable image tags
10. **Docker-Only Thinking**: Learn orchestration (Swarm/K8s)

Continuous Learning & Evolution

Docker and container ecosystem continuously evolve:

- Keep abreast of security updates
- Monitor Docker announcements and blog
- Explore emerging tools (Podman, containerd, CRI-O)
- Follow container security trends
- Participate in community
- Build real projects continuously
- Mentor others to deepen understanding

This roadmap is comprehensive and detailed enough to guide someone from absolute beginner to job-ready Docker professional. The key is consistent, hands-on practice combined with theoretical understanding. Each phase builds on the previous, creating a solid foundation for operating containerized systems in production.